

Android #9 API dan JSON Parsing

Table of Contents

<i>Android #9 API dan JSON Parsing</i>	<i>1</i>
Table of Contents	1
Mengenal apa itu API	1
Mengenal JSON (Javascript Objet Notation)	1
Alur Menampilkan Data API	2
Library Utama Praktik	2
Penerapan API dan JSON Parsing pada Android Kotlin	2
Referensi	9
Tugas Pendalaman Materi	9

Mengenal apa itu API

API (Application Programming Interface) adalah sekumpulan aturan dan protokol yang memungkinkan satu aplikasi berbicara dengan aplikasi lainnya. Dalam konteks mobile, Web API bertindak sebagai pelayan yang mengambilkan data dari dapur (Server/Database) untuk disajikan kepada pelanggan (Aplikasi Android).

- **REST (Representational State Transfer):** Arsitektur API yang paling umum digunakan dan menggunakan protokol HTTP standar.
- **SOAP :** Protokol pesan berbasis XML yang digunakan untuk pertukaran informasi antar sistem. Berbeda dengan REST yang merupakan gaya arsitektur, SOAP adalah protokol resmi dengan aturan yang sangat ketat.

Mengenal JSON (Javascript Objet Notation)

JSON adalah format pertukaran data yang ringan dan sangat mudah dibaca oleh manusia maupun mesin. JSON telah menggantikan XML karena ukurannya yang lebih kecil dan lebih cepat diproses.

Karakteristik JSON:

- Data dalam berpasangan: Terdiri dari Key (kunci) dan Value (nilai). Contoh: "nama": "Eko".
- Objek: Dibungkus dengan kurung kurawal { }.
- Array: Daftar objek yang dibungkus dengan kurung siku [].

Apa itu JSON Parsing? Parsing adalah proses mengurai teks string JSON yang diterima dari internet menjadi objek yang bisa dimengerti oleh bahasa pemrograman (dalam hal ini, diubah menjadi Kotlin Data Class).

Alur Menampilkan Data API

1. **Request (Permintaan):** Aplikasi mengirimkan permintaan HTTP (misal: GET) melalui library networking (Retrofit).
2. **Server Processing:** Server menerima permintaan, mengambil data dari database, dan mengubahnya menjadi format JSON.
3. **Response (Tanggapan):** Server mengirimkan kembali string JSON ke aplikasi dengan kode status (misal: 200 OK).
4. **Deserialization (Parsing):** Library GSON secara otomatis memetakan teks JSON tersebut ke dalam Data Class yang sudah kita buat.
5. **Data Handling:** Data yang sudah jadi objek Kotlin dikirim ke Adapter.
6. **Rendering:** RecyclerView melalui Adapter menggambar data tersebut ke dalam layout XML (item_post.xml) sehingga muncul di layar.

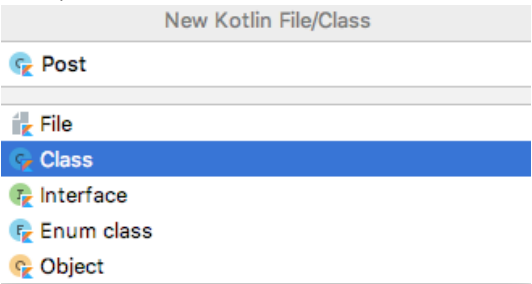
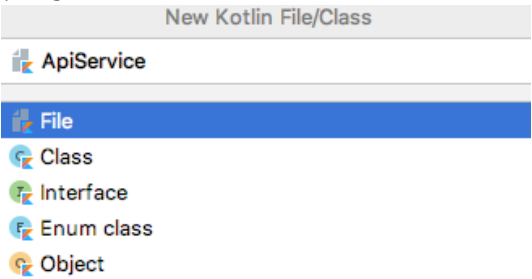
Library Utama Praktik

Library	Fungsi Utama
Retrofit	Mengelola koneksi HTTP, menangani URL, dan melakukan <i>request</i> di latar belakang (asynchronous).
GSON	"Penerjemah" yang mengubah JSON menjadi objek Kotlin secara otomatis berdasarkan nama variabel.
RecyclerView	Komponen UI paling efisien untuk menampilkan daftar data dalam jumlah banyak agar aplikasi tidak berat.

Penerapan API dan JSON Parsing pada Android Kotlin

1. Tambahkan Dependencies dengan cara buka file `build.gradle` (modul:app) dan tambahkan library Retrofit dan GSON Converter untuk menangani koneksi API dan parsing JSON.

```
// Koneksi API dan Parsing JSON (sesuaikan library dengan versi Java)
```

	<pre>implementation 'com.squareup.retrofit2:retrofit:2.6.0' implementation 'com.squareup.retrofit2:converter-gson:2.6.0'</pre>
	*jika terdapat error/force close, perhatikan versi retrofit yang digunakan, pastikan kompatibel dengan versi Java, jika masih terkendala tambahkan library berikut:
	<pre>implementation 'com.squareup.okhttp3:okhttp:3.12.0'</pre>
	Dan tambahkan kode berikut pada android { ... }:
	<pre>compileOptions { sourceCompatibility JavaVersion.VERSION_1_8 targetCompatibility JavaVersion.VERSION_1_8 }</pre>
	2. Tambahkan izin internet dengan cara buka AndroidManifest.xml dan tambahkan baris berikut di atas tag <application>
	<pre><uses-permission android:name="android.permission.INTERNET" /> <application ... </pre>
	3. Buat data class dengan nama Post sesuai dengan struktur JSON dari https://jsonplaceholder.typicode.com/posts , caranya klik kanan pada package pilih New dan pilih Kotlin File/Class:
	
	Tambahkan struktur berikut pada data class Post:
	<pre>package com.example.myapp data class Post(val userId: Int, val id: Int, val title: String, val body: String)</pre>
	4. Definisikan interface API (Retrofit) dengan cara buat file baru (Klik kanan package->New->Kotlin File/Class) dan beri nama ApiService.kt kemudian definisikan endpoint yang akan diakses.
	
	Kemudian tambahkan kode berikut pada ApiService.kt
	<pre>package com.example.myapp import retrofit2.Call import retrofit2.http.GET</pre>

```
interface ApiService {
    @GET("posts")
    fun getPosts(): Call<List<Post>>
}
```

5. Buat layout item post (item_post.xml) pada res/layout untuk menampilkan title dan body.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.cardview.widget.CardView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="8dp"
    app:cardCornerRadius="8dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <TextView
            android:id="@+id/tvPostTitle"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:textSize="18sp"
            android:textStyle="bold"
            android:textColor="@color/black" />

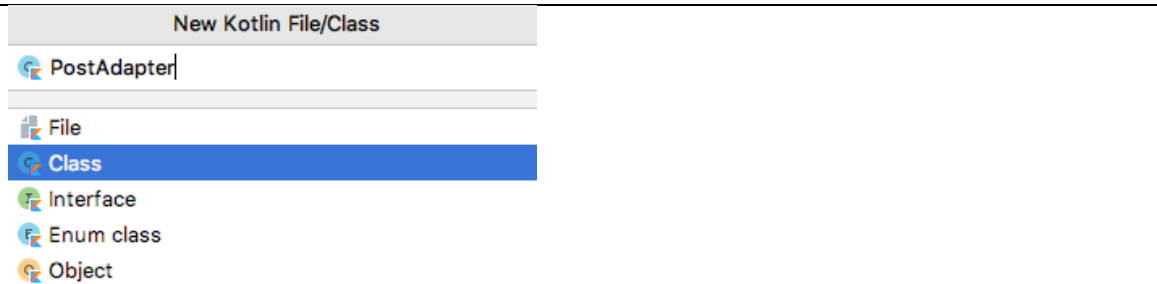
        <TextView
            android:id="@+id/tvPostBody"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="8dp"
            android:textSize="14sp" />

    </LinearLayout>
</androidx.cardview.widget.CardView>
```

6. Update layout fragment_settings.xml dengan menambahkan RecyclerView ke dalam layout Settings.

```
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/rvPost"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:listitem="@layout/item_post" />
```

7. Buat PostAdapter (mirip MahasiswaAdapter sebelumnya), namun ini disesuaikan untuk kelas Post.



Kemudian tambahkan kode berikut pada class PostAdapter:

```
package com.example.myapp

import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.TextView
import androidx.recyclerview.widget.RecyclerView

class PostAdapter(private val listPost: List<Post>) :
    RecyclerView.Adapter<PostAdapter.ViewHolder>() {

    class ViewHolder(view: View) : RecyclerView.ViewHolder(view) {
        val tvTitle : TextView = view.findViewById(R.id.tvPostTitle)
        val tvBody : TextView = view.findViewById(R.id.tvPostBody)
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
    ViewHolder {
        val view =
        LayoutInflater.from(parent.context).inflate(R.layout.item_post, parent,
        false)
        return ViewHolder(view)
    }

    override fun getItemCount(): Int {
        return listPost.size
    }

    override fun onBindViewHolder(holder: ViewHolder, position: Int) {
        val item = listPost[position]
        holder.tvTitle.text = item.title
        holder.tvBody.text = item.body
    }
}
```

8. Hubungkan API dengan Settings fragment dengan cara inisialisasi retrofit dan tampilan datanya.

```
package com.example.myapp.fragment

import android.os.Bundle
import androidx.fragment.app.Fragment
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Toast
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView
import com.example.myapp.ApiService
import com.example.myapp.Post
```

```

import com.example.myapp.PostAdapter
import com.example.myapp.R
import retrofit2.Call
import retrofit2.Callback
import retrofit2.Response
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory

// the fragment initialization parameters, e.g. ARG_ITEM_NUMBER
//private const val ARG_PARAM1 = "param1"
//private const val ARG_PARAM2 = "param2"

/**
 * A simple [Fragment] subclass.
 * Use the [Settings.newInstance] factory method to
 * create an instance of this fragment.
 */
class Settings : Fragment() {
    lateinit var rvPost: RecyclerView

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        // Inflate the layout for this fragment
        // return inflater.inflate(R.layout.fragment_settings, container,
        // false)
        val view = inflater.inflate(R.layout.fragment_settings, container,
        false)
        return view
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        //      Inisialisasi RecyclerView
        rvPost = view.findViewById(R.id.rvPost)
        rvPost.layoutManager = LinearLayoutManager(requireContext())

        //      panggil fungsi untuk mengambil data dari API
        showDataPost()
    }

    fun showDataPost() {
        //      Inisialisasi Retrofit
        val retrofit = Retrofit.Builder()
            .baseUrl("https://jsonplaceholder.typicode.com/")
            .addConverterFactory(GsonConverterFactory.create())
            .build()

        //      Hubungkan dengan Interface ApiService
        val apiService = retrofit.create(ApiService::class.java)

        //      Eksekusi Request ke API
        apiService.getPosts().enqueue(object : Callback<List<Post>> {
            override fun onFailure(call: Call<List<Post>>, t: Throwable) {
                Toast.makeText(requireContext(), "Gagal Mengambil Data:
                ${t.message}", Toast.LENGTH_SHORT).show()
            }
        })
    }

```

```

        override fun onResponse(call: Call<List<Post>>, response:
Response<List<Post>>) {
            if (response.isSuccessful) {
                val listData = response.body()
                if (listData != null) {
                    // Masukan data ke adapter dan pasang ke RecyclerVew
                    val adapter = PostAdapter(listData)
                    rvPost.adapter = adapter
                }
            }
        }
    })
}
}

```

9. Menambahkan Indikator Loading dapat dilakukan dengan memodifikasi fragment_setting.xml menjadi seperti berikut:

```

<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/pink_200"
    tools:context=".fragment.Settings">

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/rvPost"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        tools:listitem="@layout/item_post" />

    <ProgressBar
        android:id="@+id/progressBar"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:visibility="gone" />

</FrameLayout>

```

10. Kemudian tambahkan kode berikut untuk inialisasi ProgrssBar dan mengatur kapan harus VISIBLE dan kapan harus GONE, lakukan perubahan kode pada Settings.kt

```

package com.example.myapp.fragment

import android.os.Bundle
import androidx.fragment.app.Fragment
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Toast
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView
import com.example.myapp.ApiService
import com.example.myapp.Post
import com.example.myapp.PostAdapter
import com.example.myapp.R
import kotlinx.android.synthetic.main.fragment_settings.*
import retrofit2.Call
import retrofit2.Callback

```

```

import retrofit2.Response
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory

class Settings : Fragment() {
    lateinit var rvPost: RecyclerView

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        // Inflate the layout for this fragment
        return inflater.inflate(R.layout.fragment_settings, container,
false)
        val view = inflater.inflate(R.layout.fragment_settings, container,
false)
        return view
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        // Inisialisasi RecyclerView
        rvPost = view.findViewById(R.id.rvPost)
        rvPost.layoutManager = LinearLayoutManager(requireContext())

        // Panggil fungsi untuk mengambil data dari API
        showDataPost()
    }

    fun showDataPost() {
        // Tampilkan loading
        progressBar.visibility = View.VISIBLE

        // Inisialisasi Retrofit
        val retrofit = Retrofit.Builder()
            .baseUrl("https://jsonplaceholder.typicode.com/")
            .addConverterFactory(GsonConverterFactory.create())
            .build()

        // Hubungkan dengan Interface ApiService
        val apiService = retrofit.create(ApiService::class.java)

        // Eksekusi Request ke API
        apiService.getPosts().enqueue(object : Callback<List<Post>> {
            override fun onFailure(call: Call<List<Post>>, t: Throwable) {
                // Sembunyikan loading jika gagal
                progressBar.visibility = View.GONE
                Toast.makeText(requireContext(), "Gagal Mengambil Data:
${t.message}", Toast.LENGTH_SHORT).show()
            }

            override fun onResponse(call: Call<List<Post>>, response:
Response<List<Post>>) {
                // Sembunyikan loading saat respon datang
                progressBar.visibility = View.GONE

                if (response.isSuccessful) {
                    val listData = response.body()

```

```

//                                if (listData!= null) {
//                                    Masukan data ke adapter dan pasang ke RecyclerView
//                                    val adapter = PostAdapter(listData)
//                                    rvPost.adapter = adapter
//                                }
//                            }
//                        })
//                    }
//                }
//            }
//        }
//    }
}

```

11. Lakukan pengujian pada emulator



Referensi

<https://developer.android.com/>

Tugas Pendalaman Materi

Tampilkan data pada aplikasi yang anda kembangkan saat ini dari web service yang sudah tersedia secara online melalui mekanisme pertukaran data dengan arsitektur microservice (gunakan public API lain selain dari JsonPlaceholder, seperti: <https://www.omdbapi.com/>, <https://pokeapi.co/>, dsb.).